

# Use Cases von Out-of-Memory Computation im StBA

Datenbereitstellung und -analysen im Rahmen des FDZ

Oliver Hauke (C34)

25.06.2024

## Inhaltsverzeichnis

|          |                                                   |           |
|----------|---------------------------------------------------|-----------|
| <b>1</b> | <b>Einleitung</b>                                 | <b>3</b>  |
| 1.1      | Baselines . . . . .                               | 3         |
| 1.2      | Arrow . . . . .                                   | 3         |
| 1.3      | DuckDB . . . . .                                  | 4         |
| 1.4      | Parquet . . . . .                                 | 4         |
| <b>2</b> | <b>Vorbereitung</b>                               | <b>4</b>  |
| 2.1      | htop checken . . . . .                            | 4         |
| 2.2      | Pakete laden . . . . .                            | 4         |
| 2.3      | Datenablage und -Aufteilung . . . . .             | 5         |
| <b>3</b> | <b>Einführung anhand RAM-gerechter Daten</b>      | <b>6</b>  |
| 3.1      | Benchmarks . . . . .                              | 6         |
| 3.2      | Arrow für moderate Datenmengen . . . . .          | 7         |
| 3.2.1    | Vorteile . . . . .                                | 7         |
| 3.2.2    | Einschränkungen . . . . .                         | 10        |
| 3.2.2.1  | Windowed Aggregation . . . . .                    | 10        |
| 3.2.2.2  | Cross-Calculation . . . . .                       | 11        |
| 3.2.2.3  | Regression . . . . .                              | 12        |
| 3.2.2.4  | Zeilenweise Operationen . . . . .                 | 12        |
| 3.2.2.5  | Indexabhängige Operationen . . . . .              | 13        |
| 3.2.3    | Lösungen mit DuckDB . . . . .                     | 13        |
| 3.2.3.1  | Windowed Aggregation . . . . .                    | 13        |
| 3.2.3.2  | Cross-Calculation . . . . .                       | 14        |
| 3.2.3.3  | Regression . . . . .                              | 14        |
| 3.2.3.4  | Zeilenweise Operationen . . . . .                 | 15        |
| <b>4</b> | <b>Workflow Optimierungsvorschlag</b>             | <b>15</b> |
| 4.1      | Experimentieren . . . . .                         | 15        |
| 4.1.1    | Breite . . . . .                                  | 15        |
| 4.1.2    | Länge . . . . .                                   | 15        |
| 4.1.3    | Alle Chunks . . . . .                             | 16        |
| 4.2      | Reduzieren . . . . .                              | 16        |
| 4.3      | Exkurs: Optimieren . . . . .                      | 17        |
| 4.4      | Auswerten . . . . .                               | 18        |
| <b>5</b> | <b>Out-of-Memory Computation mit Arrow</b>        | <b>19</b> |
| 5.1      | Einlesen . . . . .                                | 19        |
| 5.1.1    | Schema einmalig definieren (Empfehlung) . . . . . | 19        |

|          |                                                 |           |
|----------|-------------------------------------------------|-----------|
| 5.1.2    | Verbindung zum Dateisystem herstellen . . . . . | 19        |
| 5.2      | Ein exemplarisches Experiment . . . . .         | 21        |
| 5.2.1    | Aufbereitung . . . . .                          | 21        |
| 5.2.2    | Auswertung . . . . .                            | 22        |
| 5.3      | Realer Use Case . . . . .                       | 23        |
| 5.3.1    | Auswahl . . . . .                               | 24        |
| 5.3.2    | Aufbereitung . . . . .                          | 26        |
| 5.3.3    | Anreicherung . . . . .                          | 28        |
| 5.3.4    | Auswertungsdatensatz erstellen . . . . .        | 29        |
| 5.3.5    | Auswertung . . . . .                            | 32        |
| 5.3.5.1  | Weitere Anreicherung . . . . .                  | 33        |
| 5.3.5.2  | Tabellen . . . . .                              | 33        |
| 5.3.5.3  | Modelle . . . . .                               | 35        |
| <b>6</b> | <b>Abschluss</b>                                | <b>36</b> |

# 1 Einleitung

## 1.1 Baselines

- Üblicherweise werden für Datenverarbeitung in R verwendet: `data.table`, `readr`, `r-base` und viele weitere
- die Pakete unterscheiden sich stark in Performance
- Alle o.g. Pakete können nur Daten verarbeiten, indem sie vollständig in den Arbeitsspeicher geladen werden.
- Falls der Umfang der Daten die Nähe des verfügbaren Arbeitsspeicher erreicht, kommt es zu langen Wartezeiten, Fehlern und Abbrüchen. - Dies ist insbesondere bei umfangreichen FDZ-Daten der Fall und o.g. Probleme führen i.d.R. zu Abstimmungen mit externen Forschenden.
- Hier können die Bibliotheken **Arrow** und **DuckDB** helfen, da...
  - sie auf hohe Performanz ausgelegt sind.
  - Daten nutzen können, ohne sie vollständig in den Arbeitsspeicher zu laden.

## 1.2 Arrow

**Arrow** ist eine Bibliothek in diversen Programmiersprachen, die das Einlesen und Bearbeiten von Daten effizient ermöglicht. Es bietet die Möglichkeit Datensätze zu verwenden, die nicht vollständig in den Arbeitsspeicher passen. Für Effizienzgewinne sorgen insbesondere ein geschickt gewähltes Datenformat und die optimierte Nutzung moderner Multicore-Prozessoren. Zusätzlich können sog. **Arrow Tables** mit Daten arbeiten, die sich verteilt auf einer Festplatte befinden, ohne sie vollständig in den Arbeitsspeicher zu laden.

Es folgt ausgewählte Details über **Arrow** aus

1. <https://arrow.apache.org/>
2. <https://arrow.apache.org/docs/r/index.html>

- The arrow package provides functions for reading single data files into memory, in several common formats.
- By default, calling any of these functions returns an R data frame.
- To return an **Arrow Table**, set argument `as_data_frame = FALSE`.

**Arrow Table** is a data structure than enhance interoperability and performance. Key features:

Advantages:

- Columnar Storage: column-by-column storage is advantageous for analytics.
- Minimizes memory overhead.
- Interoperability between programming languages, Standardization
- Zero-Copy Reads: avoids duplication
- Memory Mapping: maps files from disk directly into access space of the process. Only portions of the file are loaded into memory as needed.
- Out-of-core Processing: Process data that doesn't fit in Memory by breaking the data into smaller chunks.

Disadvantages:

- "Columnar Storage" ungünstig für Kalkulation für Interaktion zwischen Spalten
- "Memory Mapping" abhängig von Dateistruktur, -format, ...
- "Out-of-core Processing" + "Columnar Storage" zeilenweise Kalkulation aufwändig bzw. wenig supported. Kein nutzbarer Index für Kalkulationen.

Some Resources:

- [https://blog.djnavarro.net/posts/2022-03-04\\_data-types-in-arrow-and-r/](https://blog.djnavarro.net/posts/2022-03-04_data-types-in-arrow-and-r/)
- [https://blog.djnavarro.net/posts/2022-05-25\\_arrays-and-tables-in-arrow/](https://blog.djnavarro.net/posts/2022-05-25_arrays-and-tables-in-arrow/)
- <https://blog.streamlit.io/all-in-on-apache-arrow/>

Wer wie Wickham et al. an der Performance von Arrow zweifelt, dem sei dieser Vortrag ans Herz gelegt: Neal Richardson | Bigger Data With Ease Using Apache Arrow | RStudio (2021), <https://www.youtube.com/watch?v=zND-Wj2XPvc>. Dort wird gezeigt, dass das Paket 2-3x schneller CSV einlesen kann als `data.table`. Die dortige (high level) Erklärung führt dies auf C++ Multithreading und SIMD Optimization zurück.

### 1.3 DuckDB

DuckDB bietet ähnliche Funktionalitäten wie **Arrow** (vgl. <https://duckdb.org/docs/api/r>)

Meiner Erfahrung nach:

- **Arrow** ermöglicht das Einlesen von Daten leichter (**Duckdb** ist eher SQL-mäßig)
- **Duckdb** hat bereits mehr Funktionen für Analysen implementiert (für Daten größer als RAM: windowed Aggregation, Median/Quantile, Kreuzkalkulationen)
- Also was nimmt man? => Beides, die Packages haben eine Schnittstelle implementiert (`to_duckdb()` überführt Arrow in DuckDB und `to_arrow()` zurück).

### 1.4 Parquet

Ist ein spaltenbasiertes Datenformat, das sich effizient in div. Programmiersprachen einlesen lässt und durch automatische Kompression Festplattenspeicherbedarf reduziert. Es ist inzwischen ein beliebter Standard im Big Data Bereich und lässt sich native mit **Arrow** verbinden. Vgl. <https://parquet.apache.org/>.

## 2 Vorbereitung

### 2.1 htop checken

Vor Ausführung langfristiger oder ressourcenstrapazierenden Programme sollte im Terminal die Auslastung mit `htop` gecheckt werden.

### 2.2 Pakete laden

```
library(data.table, warn.conflicts = FALSE)
library(readr, warn.conflicts = FALSE)
library(arrow, warn.conflicts = FALSE)
library(duckdb, warn.conflicts = FALSE)

# Tidyverse Tools
library(dplyr, warn.conflicts = FALSE)
library(dbplyr, warn.conflicts = FALSE)
library(stringr, warn.conflicts = FALSE)
library(ggplot2, warn.conflicts = FALSE)

# Performance-Messung
library(tictoc, warn.conflicts = FALSE)
library(lobstr, warn.conflicts = FALSE)
library(microbenchmark, warn.conflicts = FALSE)

# Manuelles Monitoring und Parallelisierung
library(progress)
library(foreach)

arrow::set_cpu_count(36)
```

Hinweis: Arrow supported auf dem R-Server aktuell kein gzip :( (Stand 02.07.2024)

## 2.3 Datenablage und -Aufteilung

Die Datensätze sind in RAM-gerechte Blöcke aufgeteilt. Bei der Wahl von geeigneten Paketen, entsteht dadurch beim Einlesen keine Anpassung der Usability (s.u.). Datensätze, die als einzige Datei (größer als der verfügbare RAM) abgespeichert werden, lassen sie sich nur sehr unhandlich und eingeschränkt betrachten und verarbeiten.

Arrow empfiehlt:

*The most optimal partitioning layout will depend on your data, access patterns, and which systems will be reading the data. Most systems, including Arrow, should work across a range of file sizes and partitioning layouts, but there are extremes you should avoid. These guidelines can help avoid some known worst cases:*

- Avoid files smaller than 20MB and larger than 2GB.
- Avoid partitioning layouts with more than 10,000 distinct partitions.

S. <https://arrow.apache.org/docs/r/articles/dataset.html#partitioning-performance-considerations>.

```
datenpfad <- "~/fdzDaten/daten/drg/onsite_material"
```

```
datenpfade <- function(filetype = "gz", full.names = TRUE){  
  list.files(file.path(datenpfad, filetype),  
            recursive = TRUE,  
            full.names = full.names)  
}
```

```
cat(datenpfade("gz", FALSE)[1:10], sep = "\n")  
cat("...\n")  
cat(datenpfade("parquet", FALSE)[1:10], sep = "\n")  
cat("...")
```

```
#> drg2005/drg2005-part1.csv.gz  
#> drg2005/drg2005-part2.csv.gz  
#> drg2005/drg2005-part3.csv.gz  
#> drg2005/drg2005-part4.csv.gz  
#> drg2005/drg2005-part5.csv.gz  
#> drg2005/drg2005-part6.csv.gz  
#> drg2005/drg2005-part7.csv.gz  
#> drg2005/drg2005-part8.csv.gz  
#> drg2005/drg2005-part9.csv.gz  
#> drg2006/drg2006-part1.csv.gz  
#> ...  
#> berichtsjahr=2005/drg2005-part1.parquet  
#> berichtsjahr=2005/drg2005-part2.parquet  
#> berichtsjahr=2005/drg2005-part3.parquet  
#> berichtsjahr=2005/drg2005-part4.parquet  
#> berichtsjahr=2005/drg2005-part5.parquet  
#> berichtsjahr=2005/drg2005-part6.parquet  
#> berichtsjahr=2005/drg2005-part7.parquet  
#> berichtsjahr=2005/drg2005-part8.parquet  
#> berichtsjahr=2005/drg2005-part9.parquet  
#> berichtsjahr=2006/drg2006-part1.parquet  
#> ...
```

## 3 Einführung anhand RAM-gerechter Daten

Erprobt anhand der ersten 2 Mio. Beobachtungen der DRG 2005 (im RAM je nach Methode ca. 20 GB). Wenn die Daten in den RAM passen, wird für das Einlesen (klassisch) verwendet:

1. Base `read.csv`
2. `readr::read_csv`
3. `data.table::fread`
4. `vroom`
5. `arrow`
6. `duckdb`

Nr. 1: ist nicht zu empfehlen (langsam und unhandlich). Nr. 2: tlw. am Benutzerfreundlichsten aber i.d.R. langsamer als andere Optionen. Nr. 3: Schnell, flexibel und mächtig, aber benötigt eigene Einarbeitung. Nr. 4: Nr. 3 oft schneller, aber tlw. vergleichbar performant oder in Spezialfällen auch am schnellsten (s. <https://vroom.r-lib.org/articles/benchmarks.html>). Hat Benutzerfreundlichkeit aus Nr. 2. Nr. 5 / Nr. 6: s.u.

### 3.1 Benchmarks

- Fangen wir klein an: 2 Mio. Beobachtung aus 2005
- Einlesen hauptsächlich von `csv.gz`. Da die `arrow` Installation auf dem R-Server dies nicht ermöglicht, wird Parquet stattdessen genutzt.
- Name und Größe der betrachteten Dateien:

```
pfad1_gz <- datenpfade("gz")[1]
pfad1_pq <- datenpfade("parquet")[1]
pfad1_zip <- datenpfade("zip")[1]

cat(paste0(basename( pfad1_gz ), ": ", fs::file_size(pfad1_gz)))
cat("\n")
cat(paste0(basename( pfad1_pq ), ": ", fs::file_size(pfad1_pq)))
```

```
#> drg2005-part1.csv.gz: 254M
#> drg2005-part1.parquet: 170M
```

#### Ein kleiner Test (Arrow erstellt kleine Samples in Sekunden)

Wird nicht ausgeführt, aber kann für Weiterentwicklungen verwendet werden.

```
tic()
pfad1_pq <- datenpfade("parquet")[1]
da <- read_parquet(pfad1_pq, as_data_frame = FALSE)
dah <- head(da, 100)
dah |> write_csv_arrow("temp/dat.csv")
fread("temp/dat.csv") |> fwrite("temp/dat.csv.gz")
fread("temp/dat.csv") |> fwrite("temp/dat.csv.zip")
dah |> write_parquet("temp/dat.parquet")
pfad1_pq <- "temp/dat.parquet"
pfad1_gz <- "temp/dat.csv.gz"
pfad1_zip <- "temp/dat.csv.zip"
toc()
```

#### Vergleich der Einlesezeiten

```
mbm <- microbenchmark(
  read.csv2(pfad1_gz),
  readr::read_csv2(pfad1_gz, show_col_types = FALSE),
  data.table::fread(pfad1_gz),
```

```
fread_zip = data.table::fread(pfad1_gz), #fread(cmd = paste("unzip -p", pfad1_zip)),
arrow::read_parquet(pfad1_pq),
arrow::read_parquet(pfad1_pq, as_data_frame = FALSE),
times = 3, unit = "seconds"
)
```

Tabelle 1: Einlesezeiten (in sec)

| Paket     | Ausdruck                                             | min    | mean   | median | max    |
|-----------|------------------------------------------------------|--------|--------|--------|--------|
| Base-R    | read.csv2(pfad1_gz)                                  | 421.99 | 435.70 | 430.47 | 454.64 |
| Readr     | readr::read_csv2(pfad1_gz, show_col_types = FALSE)   | 306.04 | 385.96 | 415.13 | 436.70 |
| Datatable | data.table::fread(pfad1_gz)                          | 54.15  | 57.97  | 58.75  | 61.00  |
| Datatable | fread_zip                                            | 58.11  | 59.53  | 58.34  | 62.14  |
| Arrow     | arrow::read_parquet(pfad1_pq)                        | 1.43   | 2.53   | 3.02   | 3.15   |
| Arrow     | arrow::read_parquet(pfad1_pq, as_data_frame = FALSE) | 0.96   | 1.03   | 0.96   | 1.18   |

## 3.2 Arrow für moderate Datenmengen

Leider fehlt Gzip-Support auf dem R-Server (Stand 05.07.2024):

```
dat <- read_csv_arrow(pfad1_gz)
```

```
#> Error: NotImplemented: Support for codec 'gzip' not built
```

Wie in anderen Paketen üblich, kann **Arrow** Daten vollständig in den Arbeitsspeicher laden. Es wird jedoch eine spaltenbasierte Repräsentation im RAM verwendet, die De- bzw. Serialisierung reduziert und beschleunigt. Kurz gesagt, Daten sind schneller und kleiner im RAM. **Arrow** bevorzugt das Einlesen aus **Parquet**- oder **Feather**-Dateien. Beide Formate sind spaltenbasiert und lassen sich leichter **Arrow**-lesbar konvertieren als zeilenbasierte CSV-Dateien o.Ä. **Feather** ist sogar speziell eine Festplatten-Version des Arrow RAM Formats.

### 3.2.1 Vorteile

```
tic()
dat <- read_parquet(pfad1_pq)
toc()
obj_size(dat)
```

```
#> 1.443 sec elapsed
```

```
#> 4.92 GB
```

Das Einlesen geht schnell und verbraucht nur 25% des RAMs aller anderen Paketen. Aggregationen können wie gewohnt ausgeführt werden und sind ebenfalls sehr schnell.

```
tic()
dat |> group_by(kh_land) |>
  summarize(Fallzahl = n())
toc()
```

```
#> # A tibble: 16 x 2
#>   kh_land Fallzahl
#>   <int>    <int>
#> 1     1      66761
#> 2     2       8479
#> 3     3     201711
```

```
#> 4      4      13181
#> 5      5     553749
#> 6      6     114686
#> 7      7      95779
#> 8      8     136155
#> 9      9     363560
#> 10     10     12141
#> 11     11     23101
#> 12     12     97292
#> 13     13     38851
#> 14     14     116873
#> 15     15      67173
#> 16     16     90508
#> 0.059 sec elapsed
```

Aber Arrow hat zusätzlich die mächtige Option (`as_data_frame = FALSE`), die nur eine Verbindung zur Festplatte herzustellen, ohne einen Datensatz vollständig in den Arbeitsspeicher zu laden. Dies verbraucht fast gar keinen RAM und kann viele ausgewählte Datenoperationen sehr schnell durchführen, auch wenn die Daten nicht vollständig in den Arbeitsspeicher passen.

```
tic()
dat <- read_parquet(pfad1_pq, as_data_frame = FALSE)
toc()
obj_size(dat)
```

```
#> 1.111 sec elapsed
#> 284.64 kB
```

```
dat %>% select(1:20)
cat("...")
```

```
#> Table (query)
#> kh_land: int32
#> kh_rb: int32
#> kh_kreis: int32
#> kh_gem: int32
#> kh_plz: int32
#> pat_land: string
#> pat_rb: string
#> pat_kreis: string
#> pat_gem: string
#> pat_ags5: string
#> sex: string
#> alter: int32
#> typ_alter: int32
#> geb_jahr: int32
#> geb_monat: int32
#> alter_tage: int32
#> typ_geb: int32
#> aufn_anl: string
#> aufn_grd: int32
#> aufn_gew: int32
#>
#> See $.data for the source Arrow object
#> ...
```



Das Objekt ist aber nicht ohne weiteres vollständig im RAM verfügbar und außer Variablentypen wird daher auch nichts angezeigt. Für Datenverarbeitungen können bequem viele `dplyr` Verbs verwendet werden. Die Ergebnisse von `dplyr` Pipes werden aber nicht direkt kalkuliert.

```
dat |> group_by(kh_land) |>
  summarize(Fallzahl = n())
```

```
#> Table (query)
#> kh_land: int32
#> Fallzahl: int64
#>
#> See $.data for the source Arrow object
```

In der Handhabung müssen `dplyr` Pipes ausschließlich um einen Befehl `collect()` ergänzt werden, damit das Ergebnis kalkuliert und angezeigt werden. Durch diese sog. *Lazy* Computation wird vermieden, dass unnötige Rechenschritte ausgeführt werden und dadurch Wartezeiten minimiert.

```
tic()
dat |> group_by(kh_land) |>
  summarize(Fallzahl = n()) |>
  collect() |>
  print_table()
toc()
```

| kh_land | Fallzahl |
|---------|----------|
| 16      | 90508    |
| 15      | 67173    |
| 12      | 97292    |
| 3       | 201711   |
| 9       | 363560   |
| 11      | 23101    |
| 8       | 136155   |
| 5       | 553749   |
| 7       | 95779    |
| 1       | 66761    |
| 6       | 114686   |
| 14      | 116873   |
| 13      | 38851    |
| 10      | 12141    |
| 2       | 8479     |
| 4       | 13181    |

```
#> 1.284 sec elapsed
```

Falls nur ein Zwischenergebnis kalkuliert werden soll, ist empfehlenswert `collect()` durch `compute()` zu ersetzen. Dies löst die Query auf und generiert das Ergebnis als **Arrow Table**, das nicht ohne weitere Schritte bzw. Umwandlung angezeigt wird, aber perfekt geeignet ist um mit **Arrow** oder **Dplyr** weiterverarbeitet zu werden.

```
tic()
dat |> group_by(kh_land) |>
  summarize(Fallzahl = n()) |>
  compute() |>
  # Konversion um Ergebnis anzuzeigen
  tibble::as_tibble() |>
```

```
print_table()
toc()
```

| kh_land | Fallzahl |
|---------|----------|
| 8       | 136155   |
| 16      | 90508    |
| 9       | 363560   |
| 2       | 8479     |
| 7       | 95779    |
| 5       | 553749   |
| 11      | 23101    |
| 13      | 38851    |
| 12      | 97292    |
| 6       | 114686   |
| 1       | 66761    |
| 3       | 201711   |
| 14      | 116873   |
| 15      | 67173    |
| 10      | 12141    |
| 4       | 13181    |

```
#> 1.281 sec elapsed
```

### 3.2.2 Einschränkungen

Alle `dplyr` Verbs sind leider (noch) nicht in **Arrow** verfügbar. Wenn eine Operation nicht unterstützt wird, hat **Arrow** zwei Strategien:

1. Falls der Aufwand (Datenmenge, Operation) klein ist, führt **Arrow** die Operation im RAM aus und liefert das Ergebnis.
2. Falls der Aufwand (Datenmenge, Operation) groß ist, bricht **Arrow** ab.

Vgl. [https://arrow.apache.org/docs/r/articles/data\\_wrangling.html#handling-unsupported-expressions](https://arrow.apache.org/docs/r/articles/data_wrangling.html#handling-unsupported-expressions). Es folgen einige Beispiele.

**3.2.2.1 Windowed Aggregation** Lt. Doku des **Arrow** R-Pakets ([https://arrow.apache.org/docs/r/articles/data\\_wrangling.html#one-table-dplyr-verbs](https://arrow.apache.org/docs/r/articles/data_wrangling.html#one-table-dplyr-verbs)):

*Note, however, that window functions such as `ntile()` are not yet supported.*

Dies funktioniert ohne Einschränkung:

```
dat |> group_by(kh_land) |>
  summarize(mittleres_alter = round(mean(alter), 0 )) |>
  collect() |>
  print_table()
```

Dies geht nur im **Arbeitsspeicher**:

```
dat |> group_by(kh_land) |>
  filter(alter < mean(alter)) |>
  summarize(fallzahl_jung = n()) |>
  collect() |>
  print_table()
```

| kh_land | mittleres_alter |
|---------|-----------------|
| 16      | 55              |
| 15      | 51              |
| 12      | 53              |
| 3       | 53              |
| 9       | 53              |
| 11      | 60              |
| 8       | 52              |
| 5       | 53              |
| 7       | 54              |
| 1       | 51              |
| 6       | 52              |
| 14      | 54              |
| 13      | 50              |
| 10      | 53              |
| 2       | 57              |
| 4       | 63              |

#> Warning: Expression alter < mean(alter) not supported in Arrow; pulling data #> into R

| kh_land | fallzahl_jung |
|---------|---------------|
| 1       | 29303         |
| 2       | 3285          |
| 3       | 86381         |
| 4       | 5616          |
| 5       | 237404        |
| 6       | 49026         |
| 7       | 41104         |
| 8       | 58202         |
| 9       | 157697        |
| 10      | 5542          |
| 11      | 9829          |
| 12      | 41289         |
| 13      | 16558         |
| 14      | 48994         |
| 15      | 28207         |
| 16      | 36702         |

**3.2.2.2 Cross-Calculation** Kovarianz etc. zwischen Variablen ist auch nur im RAM supported.

```
dat %>%
  mutate(sex = ifelse(sex == "w", 0, 1)) |>
  summarize(cov_alter_sex = cov(alter, sex)) |>
  collect()
```

#> Warning: Error in summarize\_eval(names(exprs)[i], exprs[[i]], ctx, length(.data\$group\_by\_vars) > : #>  
Expression cov(alter, sex) is not an aggregate expression or is not supported in Arrow; pulling data into R

```
#>   cov_alter_sex
#> 1      -0.4135951
```

### 3.2.2.3 Regression Regressionen sind auch nur im RAM supported.

```
dat %>%
  {glm(alter~sex+kh_land, data = .)} %>%
  summary()
```

#> Error in list(alter, sex, kh\_land): could not find function “list”

Falls eine Operation nicht unterstützt wird, kann `collect()` vorab genutzt werden. Zumindest wenn die Kapazitäten und Performance des Systems für die Operation ausreicht.

```
dat %>% collect() %>%
  {glm(alter ~ sex + kh_land, data = .)} %>%
  summary()
```

```
#>
#> Call:
#> glm(formula = alter ~ sex + kh_land, data = .)
#>
#> Deviance Residuals:
#>      Min       1Q   Median       3Q      Max
#> -54.269  -17.664    6.336   20.116   70.812
#>
#> Coefficients:
#>              Estimate Std. Error t value Pr(>|t|)
#> (Intercept)  51.719448   0.044045 1174.234 <2e-16 ***
#> sexu        -18.026364   1.926126  -9.359 <2e-16 ***
#> sexw         1.670099   0.035749  46.717 <2e-16 ***
#> kh_land       0.054936   0.004553  12.066 <2e-16 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> (Dispersion parameter for gaussian family taken to be 634.285)
#>
#>      Null deviance: 1270093583  on 1999999  degrees of freedom
#> Residual deviance: 1268567463  on 1999996  degrees of freedom
#> AIC: 18580757
#>
#> Number of Fisher Scoring iterations: 2
```

### 3.2.2.4 Zeilenweise Operationen Zeilenweise Operationen sind auch nur im RAM supported, können aber umformuliert werden.

```
tic()
dat |>
  #collect() />
  rowwise() |>
  mutate(icd = paste(c_across(icd_nd1:icd_nd89), collapse = "; ")) |>
  select(icd) |> head()
```

#> Error in UseMethod(“rowwise”): no applicable method for ‘rowwise’ applied to an object of class “c(‘Table’, ‘ArrowTabular’, ‘ArrowObject’, ‘R6’)”

```
toc()
```

#> 0.003 sec elapsed

```

tic()
dat |>
  #collect() />
  rowwise() |>
  mutate(icd = paste(c_across(icd_nd1:icd_nd89), collapse = "; ")) |>
  select(icd) |> head()

```

#> Error in UseMethod("rowwise"): no applicable method for 'rowwise' applied to an object of class "c('Table', 'ArrowTabular', 'ArrowObject', 'R6')"

```

toc()

```

#> 0.002 sec elapsed

### 3.2.2.5 Indexabhängige Operationen

Indexabhängige Operationen sind auch nur im RAM supported.

```

tic()
dat |>
  #collect() />
  mutate(id = 1:nrow(dat)) |> select(id)

```

#> Warning: In id = 1:nrow(dat), only values of size one are recycled; pulling #> data into R

```

toc()

```

```

#>           id
#>      1:      1
#>      2:      2
#>      3:      3
#>      4:      4
#>      5:      5
#>      ---
#> 19999996: 1999996
#> 19999997: 1999997
#> 19999998: 1999998
#> 19999999: 1999999
#> 20000000: 2000000
#> 1.592 sec elapsed

```

### 3.2.3 Lösungen mit DuckDB

Duckdb lässt sich nahtlos mit Arrow kombinieren und ergänzt viele Operationen (auch out of Memory).

#### 3.2.3.1 Windowed Aggregation

gehen:

```

tic()
dat |>
  to_duckdb() |>
  group_by(kh_land) |>
  filter(alter < mean(alter)) |>
  summarize(fallzahl_jung = n()) |>
  collect() |>
  print_table()
toc()

```

#> 9.24 sec elapsed

| kh_land | fallzahl_jung |
|---------|---------------|
| 2       | 3285          |
| 10      | 5542          |
| 4       | 5616          |
| 15      | 28207         |
| 1       | 29303         |
| 12      | 41289         |
| 11      | 9829          |
| 16      | 36702         |
| 7       | 41104         |
| 8       | 58202         |
| 13      | 16558         |
| 3       | 86381         |
| 6       | 49026         |
| 5       | 237404        |
| 9       | 157697        |
| 14      | 48994         |

### 3.2.3.2 Cross-Calculation gehen:

```
tic()
dat |>
  to_duckdb() |>
  mutate(sex = ifelse(sex == "w", 0, 1)) |>
  summarize(cov_alter_sex = cov(alter, sex)) |>
  collect()
toc()
```

```
#> # A tibble: 1 x 1
#>   cov_alter_sex
#>   <dbl>
#> 1      -0.414
#> 7.42 sec elapsed
```

### 3.2.3.3 Regression gehen:

```
dat %>%
  to_duckdb() %>%
  {glm(alter~sex+kh_land, data = .)} %>%
  summary()
```

```
#>
#> Call:
#> glm(formula = alter ~ sex + kh_land, data = .)
#>
#> Deviance Residuals:
#>    Min       1Q   Median       3Q      Max
#> -54.269  -17.664    6.336   20.116   70.812
#>
#> Coefficients:
#>              Estimate Std. Error t value Pr(>|t|)
#> (Intercept)  51.719448   0.044045 1174.234  <2e-16 ***
#> sexu        -18.026364   1.926126  -9.359  <2e-16 ***
```

```
#> sexw          1.670099    0.035749    46.717    <2e-16 ***
#> kh_land        0.054936    0.004553    12.066    <2e-16 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> (Dispersion parameter for gaussian family taken to be 634.285)
#>
#> Null deviance: 1270093583  on 1999999  degrees of freedom
#> Residual deviance: 1268567463  on 1999996  degrees of freedom
#> AIC: 18580757
#>
#> Number of Fisher Scoring iterations: 2
```

**3.2.3.4 Zeilenweise Operationen** Zeilenweise Operationen gehen auch mit duckdb nicht:

```
tic()
dat |>
  to_duckdb() |>
  rowwise() |>
  mutate(icd = paste(c_across(icd_nd1:icd_nd89), collapse = "; ")) |>
  select(icd) |> head()
```

```
#> Error in UseMethod("rowwise"): no applicable method for 'rowwise' applied to an object of class
"c('tbl_duckdb_connection', 'tbl_dbi', 'tbl_sql', 'tbl_lazy', 'tbl')"
```

```
toc()
```

```
#> 4.363 sec elapsed
```

## 4 Workflow Optimierungsvorschlag

### 4.1 Experimentieren

Wir tasten uns an die Grenzen ran und erforschen die Größenordnung, die verarbeitet werden kann. Um eine Baseline zu erhalten, verwenden wir das performanteste Paket (abgesehen von **Arrow**): **Datatable**. Zunächst betrachten wir einen Chunk.

#### 4.1.1 Breite

```
tic()
drg05_head <- fread(pfad1_gz, nrow = 100)
toc()
obj_size(drg05_head)
```

```
#> 16.557 sec elapsed
```

```
#> 1.15 MB
```

#### 4.1.2 Länge

```
var_auswahl <- drg05_head %>%
  {names(.)[grep1("kh_|sex|alter|icd_hd|icd_nd", names(.))]}

cat("Anzahl betrachteter Variablen: ", length(var_auswahl), "\n")

tic()
```

```
DT <- fread(pfad1_gz, select = var_auswahl)
toc()
obj_size(DT)
```

```
#> Anzahl betrachteter Variablen: 101
#> 20.99 sec elapsed
#> 1.27 GB
```

#### 4.1.3 Alle Chunks

```
pfad_drg05 <- datenpfade("gz/drg2005")

tic()
DT_demo <- rbindlist(lapply(
  pfad_drg05[1:2], function(file)fread(file, select = var_auswahl[1:3])
))
toc()
obj_size(DT_demo)

#> 37.228 sec elapsed
#> 48.00 MB
```

## 4.2 Reduzieren

Wir definieren den Scope in den festgestellten Grenzen. Dazu:

1. Auswahl von Variablen
2. Zusammenfügen von Variablen: alle icd\_nd Codes zu einer Variablen

Ein Test:

```
#' Fügt icd_nd1 bis icd_nd89 zu einer Variable icd_nd zusammen und löscht den Rest.
collapse_nd <- function(DRG){
  icd_nd <- with(DRG, paste(
    icd_nd1, icd_nd2, icd_nd3, icd_nd4, icd_nd5, icd_nd6, icd_nd7, icd_nd8, icd_nd9,
    icd_nd10, icd_nd11, icd_nd12, icd_nd13, icd_nd14, icd_nd15, icd_nd16, icd_nd17,
    icd_nd18, icd_nd19, icd_nd20, icd_nd21, icd_nd22, icd_nd23, icd_nd24, icd_nd25,
    icd_nd26, icd_nd27, icd_nd28, icd_nd29, icd_nd30, icd_nd31, icd_nd32, icd_nd33,
    icd_nd34, icd_nd35, icd_nd36, icd_nd37, icd_nd38, icd_nd39, icd_nd40, icd_nd41,
    icd_nd42, icd_nd43, icd_nd44, icd_nd45, icd_nd46, icd_nd47, icd_nd48, icd_nd49,
    icd_nd50, icd_nd51, icd_nd52, icd_nd53, icd_nd54, icd_nd55, icd_nd56, icd_nd57,
    icd_nd58, icd_nd59, icd_nd60, icd_nd61, icd_nd62, icd_nd63, icd_nd64, icd_nd65,
    icd_nd66, icd_nd67, icd_nd68, icd_nd69, icd_nd70, icd_nd71, icd_nd72, icd_nd73,
    icd_nd74, icd_nd75, icd_nd76, icd_nd77, icd_nd78, icd_nd79, icd_nd80, icd_nd81,
    icd_nd82, icd_nd83, icd_nd84, icd_nd85, icd_nd86, icd_nd87, icd_nd88, icd_nd89,
    sep = "; "))
  icd_nd <- stringr::str_remove_all(icd_nd, "NA")
  icd_nd <- stringr::str_remove_all(icd_nd, " ;")
  icd_nd <- stringr::str_remove(icd_nd, " $")
  icd_nd <- stringr::str_replace(icd_nd, "~;$", "")

  new_vars <- (names(DRG)[!grepl("icd_nd", names(DRG))])
  DRG_red <- DRG[, ..new_vars][, `:=`(icd_nd = icd_nd)]
  DRG_red
}
```



```
tic()
DT_red <- collapse_nd(DT)
toc()
```

```
obj_sizes(DT, DT_red)
```

```
#> 38.472 sec elapsed
#> *    1.27 GB
#> * 256.90 MB
```

### Produktion: alle Chunks

```
pb <- progress_bar$new(
  format = "(:spin) [:bar] :percent [Elapsed::elapsed || Remaining: ~:eta]",
  total = length(pfad_drg05),
  clear = FALSE,      # If TRUE, clears the bar when finish
)
DT_list <- list()

for(pfad in pfad_drg05){
  DT_new <- collapse_nd(fread(pfad, select = var_auswahl, showProgress = FALSE))
  DT_list <- append(DT_list, list(DT_new))
  pb$tick()
}

DT <- rbindlist(DT_list)
rm(DT_list)##; gc(verbose = FALSE)
obj_size(DT)
```

```
#> 2.01 GB
```

## 4.3 Exkurs: Optimieren

- Parallelisierung bspw. in [https://www.blasbenito.com/post/02\\_parallelizing\\_loops\\_with\\_r/](https://www.blasbenito.com/post/02_parallelizing_loops_with_r/)
- Caching auch empfehlenswert

```
parallel::detectCores()
my.cluster <- parallel::makeCluster(6)
doParallel::registerDoParallel(my.cluster)
```

```
tic()
```

```
DT_list <- foreach (pfad = pfad_drg05) %dopar% {
  collapse_nd(
    data.table::fread(pfad, select = var_auswahl, showProgress = FALSE, nThread = 6)
  )
}
```

```
toc()
```

```
parallel::stopCluster(cl = my.cluster)
```

```
# 165.665 sec elapsed = 2.45 min
```

```
DT <- rbindlist(DT_list)
```

## 4.4 Auswerten

Bspw. Verteilung von Anzahl der Nebendiagnosen.

```
tic()
DT[, `:=`(anzahl_nebendiagnosen = str_count(icd_nd, ";"), jung = (alter < 60))]
```

| anzahl_nebendiagnosen | jung     | count   |
|-----------------------|----------|---------|
| 1                     | 0 FALSE  | 495938  |
| 2                     | 0 TRUE   | 1947319 |
| 3                     | 1 TRUE   | 1764529 |
| 4                     | 1 FALSE  | 693539  |
| 5                     | 2 TRUE   | 1429264 |
| 113                   | 56 TRUE  | 2       |
| 114                   | 57 FALSE | 1       |
| 115                   | 57 TRUE  | 1       |
| 116                   | 58 TRUE  | 1       |
| 117                   | 61 TRUE  | 1       |

```
DT_out <- DT[order(anzahl_nebendiagnosen), .(count = .N), by = list(anzahl_nebendiagnosen, jung)]
#DT %>% ggplot(aes(x = anzahl_nebendiagnosen, fill = jung)) + geom_bar()
#DT_out <- DT[anzahl_nebendiagnosen > 50]
DT_out
toc()
# 25.573 sec elapsed
```

```
#>      anzahl_nebendiagnosen  jung  count
#> 1:                        0 FALSE 495938
#> 2:                        0  TRUE 1947319
#> 3:                        1  TRUE 1764529
#> 4:                        1 FALSE 693539
#> 5:                        2  TRUE 1429264
#> ---
#> 113:                      56  TRUE      2
#> 114:                      57 FALSE      1
#> 115:                      57  TRUE      1
#> 116:                      58  TRUE      1
#> 117:                      61  TRUE      1
#> 6.904 sec elapsed
```

```
drg05_sel[, .(count = sum(count)), by = kh_land]
```

```
#> Error in eval(expr, envir, enclos): object 'drg05_sel' not found
```

```
drg05_sel[order(kh_land), .(anzahl = .N), by = kh_land]
```

```
#> Error in eval(expr, envir, enclos): object 'drg05_sel' not found
```

```
drg05_sel %>% mutate(jung = alter < 60) %>%
  group_by(kh_land, sex, jung) %>%
  summarise(anzahl = n()) %>%
  group_by(kh_land) %>%
  mutate(anteil_land = anzahl / sum(anzahl) * 100)
```

```
#> Error in mutate(., jung = alter < 60): object 'drg05_sel' not found
```

```
drg05_sel %>%
  group_by(kh_land) %>%
  summarise(anzahl = n())
```

```
#> Error in group_by(., kh_land): object 'drg05_sel' not found
```

```
drg05_sel[order(kh_land), .(anzahl = .N), by = kh_land]
```

```
#> Error in eval(expr, envir, enclos): object 'drg05_sel' not found
```

## 5 Out-of-Memory Computation mit Arrow

Der in 3. beschriebene Prozess bietet schon effiziente Analysen, aber ist gegenüber **Arrow** stark manuell und daher...

1. aufwändig,
2. komplex,
3. spezifisch (im Anwendungsfall),
4. benötigt Abstimmung, Schleifen mit GaWi's und "Feingefühl" (bspw. Steuerung der Cores)

Daher die Empfehlung: **Arrow** vereinfacht und optimiert das vorgestellte Vorgehen. Auch wenn noch nicht alles voll ausgereift durch entwickelt ist (und wir auch nicht die neuste Version haben).

### 5.1 Einlesen

#### 5.1.1 Schema einmalig definieren (Empfehlung)

**Achtung:** Es kann Euch das Folgende begegnen:

```
#Error in `compute.arrow_dplyr_query()` :  
# Invalid: Failed to parse value:  
#Backtrace:  
# 1. ... %>% str()  
# 4. arrow:::collect.arrow_dplyr_query(.)  
# 5. arrow:::compute.arrow_dplyr_query(x)
```

```
#Error in compute.arrow_dplyr_query(x) :
```

Dagegen hilft es, Variablentypen vorab in Form eines **Schema** zu definieren. Dies geht sicher auch "automatisierter" als hier dargestellt:

```
schema <- schema(  
  berichtsjahr = int32(), int32(), kh_rb = int32(),  
  kh_kreis = int32(), kh_gem = int32(), kh_plz = int32(),  
  kh_typ_gem3 = int32(), pat_land = utf8(), pat_rb = utf8()  
  #, ...  
)  
  
qs::qsave(schema0$code(), "cache/drg_schema_test.qs")
```

Das einmalig erzeugte Schema lässt sich als R-Objekt abspeichern und für jedes Projekt wiederverwenden. Das klassische **rds** ist dafür i.d.R. nicht zu empfehlen, da das Paket **qs** die gleiche Funktionalität deutlich schneller und Festplattenschonender umsetzt (vgl. <https://cran.r-project.org/web/packages/qs/vignettes/vignette.html>). Die alternative **fs** kann schneller für Daten sein, aber ist nicht wie **qs** und **rds** für alle R-Objekte universell einsetzbar.

#### 5.1.2 Verbindung zum Dateisystem herstellen

Arrow kommt damit klar, dass Datensätze über mehrere Dateien verteilt vorliegen. Es muss lediglich der Ordnerpfad spezifiziert werden und wenn eine Ordnerstruktur vorliegt, in welche Variable die Ordnerinfo abgelegt werden soll (hier: **berichtsjahr**). Im vorliegenden Dateisystem ist die Struktur wie nachfolgend aufgebaut:

- berichtsjahr=2005/drg2005-part1.parquet
- berichtsjahr=2005/drg2005-part2.parquet
- berichtsjahr=2005/drg2005-part3.parquet
- berichtsjahr=2005/drg2005-part4.parquet
- berichtsjahr=2005/drg2005-part5.parquet

- berichtsjahr=2005/drg2005-part6.parquet
- berichtsjahr=2005/drg2005-part7.parquet
- berichtsjahr=2005/drg2005-part8.parquet
- berichtsjahr=2005/drg2005-part9.parquet
- berichtsjahr=2006/drg2006-part1.parquet
- berichtsjahr=2006/drg2006-part...parquet
- ...
- berichtsjahr=2022/drg2022-part...parquet

Die Angabe des Schema's ist optional, sorgt aber für mehr Stabilität.

```
tic()

schema_drg <- eval(qs::qread("cache/drg_schema.qs"))
ordner_drg <- "/daten1/home/hauke-o/fdzDaten/daten/drg/onsite_material/parquet/"

da <- open_dataset(ordner_drg, schema=schema_drg, partitioning = "berichtsjahr")

toc()
obj_size(da)
```

```
#> 1.243 sec elapsed
#> 262.61 kB
```

Das ging schnell und ist sehr klein. Sind damit wirklich die Daten verfügbar? Ein Test (Fallzahl pro Berichtsjahr) ergibt:

```
tic()
auswertung1 <- da |> group_by(berichtsjahr) |>
  summarize(Fallzahl = n()) |>
  collect()
auswertung1
toc()
```

```
#> # A tibble: 18 x 2
#>   berichtsjahr Fallzahl
#>   <int>      <int>
#> 1      2005 16071846
#> 2      2006 16230407
#> 3      2007 16600472
#> 4      2008 16924180
#> 5      2009 17191063
#> 6      2010 17434600
#> 7      2011 17708910
#> 8      2012 17976447
#> 9      2013 18133711
#> 10     2014 18531819
#> 11     2015 18665238
#> 12     2016 18961650
#> 13     2017 18901222
#> 14     2018 18754571
#> 15     2019 18825654
#> 16     2020 16359312
#> 17     2021 16233822
#> 18     2022 16253085
#> 9.417 sec elapsed
```

## 5.2 Ein exemplarisches Experiment

### 5.2.1 Aufbereitung

icd\_nd Codes werden zusammengefasst.

```
tic()
da_out05 <- da %>%
  filter(berichtsjaahr == 2005) %>%
  select(starts_with("kh_"), sex, alter, icd_hd, starts_with("icd_nd")) %>%

  mutate(icd_nd = paste(
    icd_nd1, icd_nd2, icd_nd3, icd_nd4, icd_nd5, icd_nd6, icd_nd7, icd_nd8,
    icd_nd9, icd_nd10, icd_nd11, icd_nd12, icd_nd13, icd_nd14, icd_nd15,
    icd_nd16, icd_nd17, icd_nd18, icd_nd19, icd_nd20, icd_nd21, icd_nd22,
    icd_nd23, icd_nd24, icd_nd25, icd_nd26, icd_nd27, icd_nd28, icd_nd29,
    icd_nd30, icd_nd31, icd_nd32, icd_nd33, icd_nd34, icd_nd35, icd_nd36,
    icd_nd37, icd_nd38, icd_nd39, icd_nd40, icd_nd41, icd_nd42, icd_nd43,
    icd_nd44, icd_nd45, icd_nd46, icd_nd47, icd_nd48, icd_nd49, icd_nd50,
    icd_nd51, icd_nd52, icd_nd53, icd_nd54, icd_nd55, icd_nd56, icd_nd57,
    icd_nd58, icd_nd59, icd_nd60, icd_nd61, icd_nd62, icd_nd63, icd_nd64,
    icd_nd65, icd_nd66, icd_nd67, icd_nd68, icd_nd69, icd_nd70, icd_nd71,
    icd_nd72, icd_nd73, icd_nd74, icd_nd75, icd_nd76, icd_nd77, icd_nd78,
    icd_nd79, icd_nd80, icd_nd81, icd_nd82, icd_nd83, icd_nd84, icd_nd85,
    icd_nd86, icd_nd87, icd_nd88, icd_nd89
    , sep = "; "
  ),
  .keep = "unused") %>%

  mutate(icd_nd =
    str_remove_all(icd_nd, "NA") %>%
    str_remove_all(" ;") %>%
    str_remove_all("$") %>%
    str_replace("^;$", "")
  ) %>%
  mutate(anzahl_nebendiagnosen = str_count(icd_nd, ";"))

da_out05 %>% head() %>% collect() %>% str()
```

```
toc()
# 2.15 sec elapsed
obj_size(da_out05)
# 493.41 kB
```

```
#> tibble [6 x 11] (S3: tbl_df/tbl/data.frame)
#> $ kh_land      : int [1:6] 5 5 5 5 5 5
#> $ kh_rb        : int [1:6] 9 9 9 9 9 9
#> $ kh_kreis     : int [1:6] 78 78 78 78 78 78
#> $ kh_gem       : int [1:6] 40 40 40 40 40 40
#> $ kh_plz       : int [1:6] 59368 59368 59368 59368 59368 59368
#> $ kh_typ_gem3   : int [1:6] NA NA NA NA NA NA
#> $ sex          : chr [1:6] "m" "w" "w" "w" ...
#> $ alter        : int [1:6] 62 74 81 83 78 63
#> $ icd_hd       : chr [1:6] "I2511" "I634" "K5790" "R001" ...
#> $ icd_nd       : chr [1:6] "I209;" "F03; I64; I652;" "I071; I1090; Z954;" "E876; I351; N390
#> $ anzahl_nebendiagnosen: int [1:6] 1 3 3 3 2 1
```

```
#> 1.824 sec elapsed
#> 501.07 kB
```

### 5.2.2 Auswertung

Anzahl der Nebendiagnosen in Abhängigkeit des Alters im Jahr 2005.

```
tic()
auswertung1 <- da_out05 |> mutate(jung = (alter < 60)) |>
  group_by(anzahl_nebendiagnosen) |>
  summarize(anzahl_gesamt = n(),
            anzahl_jung = sum(jung == TRUE),
            anteil_jung = round(anzahl_jung/anzahl_gesamt * 100, 2)
  ) |> arrange(anzahl_nebendiagnosen) |>
  collect()
toc()
obj_size(auswertung1)

print(auswertung1, n = 100)
```

```
#> 17.786 sec elapsed
#> 2.93 kB
#> # A tibble: 60 x 4
#>   anzahl_nebendiagnosen anzahl_gesamt anzahl_jung anteil_jung
#>   <int>          <int>      <int>      <dbl>
#> 1             0      2443257    1947319    79.7
#> 2             1      2458068    1764529    71.8
#> 3             2      2315996    1429264    61.7
#> 4             3      1968379     994283    50.5
#> 5             4      1607058     666206    41.5
#> 6             5      1266070     438431    34.6
#> 7             6       977713     286113    29.3
#> 8             7       744304     187833    25.2
#> 9             8       561415     124624    22.2
#> 10            9       423895      84423    19.9
#> 11           10       319147      57959    18.2
#> 12           11       238909      40580    17.0
#> 13           12       178937      29371    16.4
#> 14           13       134802      21368    15.8
#> 15           14       101627      15861    15.6
#> 16           15        76960      11962    15.5
#> 17           16        58355       9043    15.5
#> 18           17        44089       6989    15.8
#> 19           18        34174       5609    16.4
#> 20           19        26210       4446    17.0
#> 21           20        20075       3555    17.7
#> 22           21        15304       2881    18.8
#> 23           22        11595       2234    19.3
#> 24           23         8935       1775    19.9
#> 25           24         7235       1440    19.9
#> 26           25         5563       1190    21.4
#> 27           26         4429        920    20.8
#> 28           27         3567        813    22.8
#> 29           28         2866        657    22.9
#> 30           29         2283        509    22.3
```

```

#> 31          30          1849          461          24.9
#> 32          31          1496          382          25.5
#> 33          32          1279          337          26.4
#> 34          33          1001          251          25.1
#> 35          34           852          244          28.6
#> 36          35           656          190          29.0
#> 37          36           607          178          29.3
#> 38          37           429          137          31.9
#> 39          38           381          114          29.9
#> 40          39           332          110          33.1
#> 41          40           272           89          32.7
#> 42          41           221           75          33.9
#> 43          42           208           72          34.6
#> 44          43           175           58          33.1
#> 45          44           141           55          39.0
#> 46          45           107           37          34.6
#> 47          46           102           37          36.3
#> 48          47            86           37          43.0
#> 49          48           102           48          47.1
#> 50          49           106           47          44.3
#> 51          50            83           40          48.2
#> 52          51            66           29          43.9
#> 53          52            41           18          43.9
#> 54          53            19            9          47.4
#> 55          54             9            3          33.3
#> 56          55             2            2          100
#> 57          56             3            2          66.7
#> 58          57             2            1           50
#> 59          58             1            1          100
#> 60          61             1            1          100

```

D.h. alte Menschen haben tendentiell mehr Nebendiagnosen.

```

rm(list = ls())
gc()

```

```

#>          used (Mb) gc trigger (Mb)    max used (Mb)
#> Ncells 3225845 172.3  14803804 790.7  18504754  988.3
#> Vcells 14034543 107.1   785214269 5990.8 6593431809 50303.9

```

### 5.3 Realer Use Case

Hierbei handelt es sich um einen Teil eines realen Forschungsprojekts. Wir betrachten

- Beobachtungen: Alle ab DRG 2011
- Variablen: Ort, sex, alter, tage, entl\_grd, ik, icd\_hd\_xx, icd\_nd\_xx, ops\_ko\_xx

Es werden Einflussfaktoren gewisser Diagnosen untersucht.

```

tic()

schema_drg <- eval(qs::qread("cache/drg_schema.qs"))
ordner_drg <- "/daten1/home/hauke-o/fdzDaten/daten/drg/onsite_material/parquet/"

da <- open_dataset(ordner_drg, schema=schema_drg, partitioning = "berichtsjaehr")

```

```
toc()
obj_size(da)
```

```
#> 0.504 sec elapsed
#> 262.61 kB
```

### 5.3.1 Auswahl

```
tic()

auswahl <- function(da){

  da %>% filter(berichtsjaar > 2010) %>%
  select(
    berichtsjaar,
    kh_land, kh_typ_gem3,
    sex, alter, aufn_anl, cm_vol,
    starts_with(c("icd_hd", "icd_nd", "ops_ko")),
    tage, entl_grd, ik
  )
}

da %>% auswahl() %>% head() %>% collect() %>% str()

toc()
```

```
#3.917 sec elapsed
```

```
#> tibble [6 x 203] (S3: tbl_df/tbl/data.frame)
#> $ berichtsjaar: int [1:6] 2011 2011 2011 2011 2011 2011
#> $ kh_land      : int [1:6] 9 14 14 14 14 14
#> $ kh_typ_gem3  : int [1:6] 1 1 1 1 1 1
#> $ sex          : chr [1:6] "w" "m" "w" "m" ...
#> $ alter        : int [1:6] 0 64 80 72 70 46
#> $ aufn_anl     : chr [1:6] "N" "E" "A" "E" ...
#> $ cm_vol       : num [1:6] 2356 3294 4115 3902 47047 ...
#> $ icd_hd3      : chr [1:6] "J21" "I47" "I21" "I49" ...
#> $ icd_hd4      : chr [1:6] "J210" "I471" "I211" "I493" ...
#> $ icd_hd       : chr [1:6] "J210" "I471" "I211" "I493" ...
#> $ icd_nd1      : chr [1:6] "A080" "" "I2513" "I451" ...
#> $ icd_nd2      : chr [1:6] "E86" "" "E6600" "I1000" ...
#> $ icd_nd3      : chr [1:6] "J9600" "" "I1000" "Z966" ...
#> $ icd_nd4      : chr [1:6] "" "" "E785" "" ...
#> $ icd_nd5      : chr [1:6] "" "" "I2728" "" ...
#> $ icd_nd6      : chr [1:6] "" "" "K296" "" ...
#> $ icd_nd7      : chr [1:6] "" "" "R739" "" ...
#> $ icd_nd8      : chr [1:6] "" "" "Z966" "" ...
#> $ icd_nd9      : chr [1:6] "" "" "" "" ...
#> $ icd_nd10     : chr [1:6] "" "" "" "" ...
#> $ icd_nd11     : chr [1:6] "" "" "" "" ...
#> $ icd_nd12     : chr [1:6] "" "" "" "" ...
#> $ icd_nd13     : chr [1:6] "" "" "" "" ...
#> $ icd_nd14     : chr [1:6] "" "" "" "" ...
#> $ icd_nd15     : chr [1:6] "" "" "" "" ...
```



[illegible]

```

#> $ icd_nd70 : chr [1:6] "" "" "" "" ...
#> $ icd_nd71 : chr [1:6] "" "" "" "" ...
#> $ icd_nd72 : chr [1:6] "" "" "" "" ...
#> $ icd_nd73 : chr [1:6] "" "" "" "" ...
#> $ icd_nd74 : chr [1:6] "" "" "" "" ...
#> $ icd_nd75 : chr [1:6] "" "" "" "" ...
#> $ icd_nd76 : chr [1:6] "" "" "" "" ...
#> $ icd_nd77 : chr [1:6] "" "" "" "" ...
#> $ icd_nd78 : chr [1:6] "" "" "" "" ...
#> $ icd_nd79 : chr [1:6] NA NA NA NA ...
#> $ icd_nd80 : chr [1:6] NA NA NA NA ...
#> $ icd_nd81 : chr [1:6] NA NA NA NA ...
#> $ icd_nd82 : chr [1:6] NA NA NA NA ...
#> $ icd_nd83 : chr [1:6] NA NA NA NA ...
#> $ icd_nd84 : chr [1:6] NA NA NA NA ...
#> $ icd_nd85 : chr [1:6] NA NA NA NA ...
#> $ icd_nd86 : chr [1:6] NA NA NA NA ...
#> $ icd_nd87 : chr [1:6] NA NA NA NA ...
#> $ icd_nd88 : chr [1:6] NA NA NA NA ...
#> $ icd_nd89 : chr [1:6] NA NA NA NA ...
#> [list output truncated]
#> 2.028 sec elapsed

```

### 5.3.2 Aufbereitung

Wie zuvor werden `icd_nd` Codes zusammengefasst. Zusätzlich wird analog mit `ops_ko` verfahren.

```

tic()

aufbereitung <- function(da){
  da %>%

  mutate(icd_nd = paste(
    icd_nd1, icd_nd2, icd_nd3, icd_nd4, icd_nd5, icd_nd6, icd_nd7, icd_nd8,
    icd_nd9, icd_nd10, icd_nd11, icd_nd12, icd_nd13, icd_nd14, icd_nd15,
    icd_nd16, icd_nd17, icd_nd18, icd_nd19, icd_nd20, icd_nd21, icd_nd22,
    icd_nd23, icd_nd24, icd_nd25, icd_nd26, icd_nd27, icd_nd28, icd_nd29,
    icd_nd30, icd_nd31, icd_nd32, icd_nd33, icd_nd34, icd_nd35, icd_nd36,
    icd_nd37, icd_nd38, icd_nd39, icd_nd40, icd_nd41, icd_nd42, icd_nd43,
    icd_nd44, icd_nd45, icd_nd46, icd_nd47, icd_nd48, icd_nd49, icd_nd50,
    icd_nd51, icd_nd52, icd_nd53, icd_nd54, icd_nd55, icd_nd56, icd_nd57,
    icd_nd58, icd_nd59, icd_nd60, icd_nd61, icd_nd62, icd_nd63, icd_nd64,
    icd_nd65, icd_nd66, icd_nd67, icd_nd68, icd_nd69, icd_nd70, icd_nd71,
    icd_nd72, icd_nd73, icd_nd74, icd_nd75, icd_nd76, icd_nd77, icd_nd78,
    icd_nd79, icd_nd80, icd_nd81, icd_nd82, icd_nd83, icd_nd84, icd_nd85,
    icd_nd86, icd_nd87, icd_nd88, icd_nd89,
    sep = "; "
  ) %>%
  str_remove_all("NA") %>%
  str_remove_all(" ;") %>%
  str_remove_all(" $") %>%
  str_replace("^;$", "")
  , .keep = "unused"

) %>%

```

```

# ops_ko-----
mutate(ops_ko = paste(
  ops_ko1, ops_ko2, ops_ko3, ops_ko4, ops_ko5, ops_ko6, ops_ko7, ops_ko8,
  ops_ko9, ops_ko10, ops_ko11, ops_ko12, ops_ko13, ops_ko14, ops_ko15,
  ops_ko16, ops_ko17, ops_ko18, ops_ko19, ops_ko20, ops_ko21, ops_ko22,
  ops_ko23, ops_ko24, ops_ko25, ops_ko26, ops_ko27, ops_ko28, ops_ko29,
  ops_ko30, ops_ko31, ops_ko32, ops_ko33, ops_ko34, ops_ko35, ops_ko36,
  ops_ko37, ops_ko38, ops_ko39, ops_ko40, ops_ko41, ops_ko42, ops_ko43,
  ops_ko44, ops_ko45, ops_ko46, ops_ko47, ops_ko48, ops_ko49, ops_ko50,
  ops_ko51, ops_ko52, ops_ko53, ops_ko54, ops_ko55, ops_ko56, ops_ko57,
  ops_ko58, ops_ko59, ops_ko60, ops_ko61, ops_ko62, ops_ko63, ops_ko64,
  ops_ko65, ops_ko66, ops_ko67, ops_ko68, ops_ko69, ops_ko70, ops_ko71,
  ops_ko72, ops_ko73, ops_ko74, ops_ko75, ops_ko76, ops_ko77, ops_ko78,
  ops_ko79, ops_ko80, ops_ko81, ops_ko82, ops_ko83, ops_ko84, ops_ko85,
  ops_ko86, ops_ko87, ops_ko88, ops_ko89, ops_ko90, ops_ko91, ops_ko92,
  ops_ko93, ops_ko94, ops_ko95, ops_ko96, ops_ko97, ops_ko98, ops_ko99,
  ops_ko100, ops_ko101,
  sep = "; "
) %>%
  str_remove_all("NA") %>%
  str_remove_all(" ;") %>%
  str_remove_all(" $") %>%
  str_replace("^;$", "")
  , .keep = "unused"
)
}

da %>% auswahl() %>% aufbereitung() %>%
  head() %>% collect() %>% str()

toc()

```

```

#> tibble [6 x 15] (S3: tbl_df/tbl/data.frame)
#> $ berichtsjaar: int [1:6] 2011 2011 2011 2011 2011 2011
#> $ kh_land      : int [1:6] 8 8 8 3 8 8
#> $ kh_typ_gem3  : int [1:6] 2 2 2 2 2 2
#> $ sex          : chr [1:6] "w" "m" "m" "w" ...
#> $ alter        : int [1:6] 38 44 83 9 0 75
#> $ aufn_anl     : chr [1:6] "E" "E" "N" "N" ...
#> $ cm_vol       : num [1:6] 3228 1313 2700 694 769 ...
#> $ icd_hd3      : chr [1:6] "S83" "K60" "K80" "H10" ...
#> $ icd_hd4      : chr [1:6] "S835" "K603" "K801" "H101" ...
#> $ icd_hd       : chr [1:6] "S8350" "K603" "K8010" "H101" ...
#> $ tage         : int [1:6] 2 3 4 1 3 2
#> $ entl_grd     : int [1:6] 1 1 1 1 1 1
#> $ ik           : int [1:6] 260830026 260830026 260830026 260342218 260830026 260830026
#> $ icd_nd       : chr [1:6] "M179; T784;" "K611;" "Z921; E890; I1090; E1190; I4811; E274; Z966; Z950;
#> $ ops_ko       : chr [1:6] "58109h; 58134;" "549112;" "551112;" "" ...
#> 2.378 sec elapsed

```

Neue Variablen werden generieren, in diesem Fall werde relevante Diagnosen markiert. Hierbei ist das Fachwissen der Forschenden wesentlich erforderlich.

28

```
#> 6.072 sec elapsed
```

### 5.3.4 Auswertungsdatensatz erstellen

Sollte als Job ausgeführt werden:

```
da %>% auswahl() %>%  
  aufbereitung() %>%  
  anreicherung() %>%  
  filter(ops_immuneadsorption | ops_plasmasep) %>%  
  write_parquet("cache/auswertungsdaten_neu.parquet")
```

Ein exemplarisches Skript für den Job Launcher:

```
source("07-mustersyntax-arrow-drg-roh.R")
```

```
#> # Packages ----  
#>  
#> library(arrow, warn.conflicts = FALSE)  
#> library(duckdb, warn.conflicts = FALSE)  
#> library(dplyr, warn.conflicts = FALSE)  
#> library(dbplyr, warn.conflicts = FALSE)  
#> library(stringr, warn.conflicts = FALSE)  
#> library(data.table, warn.conflicts = FALSE)  
#>  
#> arrow::set_cpu_count(12)  
#>  
#> # Einlesen ----  
#>  
#> base_pfad      <- "/daten1/home/hauke-o/fdzDaten/daten/drg"  
#> schema_pfad    <- file.path(base_pfad, "metadaten/schema_drg.qs")  
#> datenordner_pfad <- file.path(base_pfad, "onsite_material/parquet")  
#>  
#> schema <- eval(qs::qread(schema_pfad))  
#> ordner_drg <- "/daten1/home/hauke-o/fdzDaten/daten/drg/onsite_material/parquet/"  
#>  
#> da <- open_dataset(datenordner_pfad, schema=schema, partitioning = "berichtsjaar")  
#>  
#> # Vorbereitung ----  
#>  
#> auswahl <- function(da){  
#>  
#>   da |> filter(berichtsjaar > 2010) |>  
#>     select(  
#>       berichtsjaar,  
#>       kh_land, kh_typ_gem3,  
#>       sex, alter, aufn_anl, cm_vol,  
#>       starts_with(c("icd_hd","icd_nd", "ops_ko")),  
#>       tage, entl_grd, ik  
#>     )  
#> }  
#>  
#> aufbereitung <- function(da){  
#>   da |>  
#>  
#>     mutate(icd_nd = paste(  

```

```

#>     icd_nd1, icd_nd2, icd_nd3, icd_nd4, icd_nd5, icd_nd6, icd_nd7, icd_nd8,
#>     icd_nd9, icd_nd10, icd_nd11, icd_nd12, icd_nd13, icd_nd14, icd_nd15,
#>     icd_nd16, icd_nd17, icd_nd18, icd_nd19, icd_nd20, icd_nd21, icd_nd22,
#>     icd_nd23, icd_nd24, icd_nd25, icd_nd26, icd_nd27, icd_nd28, icd_nd29,
#>     icd_nd30, icd_nd31, icd_nd32, icd_nd33, icd_nd34, icd_nd35, icd_nd36,
#>     icd_nd37, icd_nd38, icd_nd39, icd_nd40, icd_nd41, icd_nd42, icd_nd43,
#>     icd_nd44, icd_nd45, icd_nd46, icd_nd47, icd_nd48, icd_nd49, icd_nd50,
#>     icd_nd51, icd_nd52, icd_nd53, icd_nd54, icd_nd55, icd_nd56, icd_nd57,
#>     icd_nd58, icd_nd59, icd_nd60, icd_nd61, icd_nd62, icd_nd63, icd_nd64,
#>     icd_nd65, icd_nd66, icd_nd67, icd_nd68, icd_nd69, icd_nd70, icd_nd71,
#>     icd_nd72, icd_nd73, icd_nd74, icd_nd75, icd_nd76, icd_nd77, icd_nd78,
#>     icd_nd79, icd_nd80, icd_nd81, icd_nd82, icd_nd83, icd_nd84, icd_nd85,
#>     icd_nd86, icd_nd87, icd_nd88, icd_nd89,
#>     sep = "; "
#> ) |>
#>     str_remove_all("NA") |>
#>     str_remove_all(" ;") |>
#>     str_remove_all(" $") |>
#>     str_replace("^;$", "")
#> , .keep = "unused"
#>
#> ) |>
#> # ops_ko-----
#> mutate(ops_ko = paste(
#>     ops_ko1, ops_ko2, ops_ko3, ops_ko4, ops_ko5, ops_ko6, ops_ko7, ops_ko8,
#>     ops_ko9, ops_ko10, ops_ko11, ops_ko12, ops_ko13, ops_ko14, ops_ko15,
#>     ops_ko16, ops_ko17, ops_ko18, ops_ko19, ops_ko20, ops_ko21, ops_ko22,
#>     ops_ko23, ops_ko24, ops_ko25, ops_ko26, ops_ko27, ops_ko28, ops_ko29,
#>     ops_ko30, ops_ko31, ops_ko32, ops_ko33, ops_ko34, ops_ko35, ops_ko36,
#>     ops_ko37, ops_ko38, ops_ko39, ops_ko40, ops_ko41, ops_ko42, ops_ko43,
#>     ops_ko44, ops_ko45, ops_ko46, ops_ko47, ops_ko48, ops_ko49, ops_ko50,
#>     ops_ko51, ops_ko52, ops_ko53, ops_ko54, ops_ko55, ops_ko56, ops_ko57,
#>     ops_ko58, ops_ko59, ops_ko60, ops_ko61, ops_ko62, ops_ko63, ops_ko64,
#>     ops_ko65, ops_ko66, ops_ko67, ops_ko68, ops_ko69, ops_ko70, ops_ko71,
#>     ops_ko72, ops_ko73, ops_ko74, ops_ko75, ops_ko76, ops_ko77, ops_ko78,
#>     ops_ko79, ops_ko80, ops_ko81, ops_ko82, ops_ko83, ops_ko84, ops_ko85,
#>     ops_ko86, ops_ko87, ops_ko88, ops_ko89, ops_ko90, ops_ko91, ops_ko92,
#>     ops_ko93, ops_ko94, ops_ko95, ops_ko96, ops_ko97, ops_ko98, ops_ko99,
#>     ops_ko100, ops_ko101,
#>     sep = "; "
#> ) |>
#>     str_remove_all("NA") |>
#>     str_remove_all(" ;") |>
#>     str_remove_all(" $") |>
#>     str_replace("^;$", "")
#> , .keep = "unused"
#>
#> )
#> }
#>
#> anreicherung <- function(da){
#>   da |> mutate(
#>
#>     icd_ANCA_overlap = grepl("^M313|^M317", icd_nd),

```

```

#>   icd_bleeding = grepl("^D62|^D699|^H356|^H431|^I609|^I610|^I613|^I614|^I615|^I619|^I620|^J942|^K2
#>   ops_immuneadsorption = grepl("^8821", ops_ko),
#>   ops_plasmasep       = grepl("^8820", ops_ko),
#>   ops_transfusion     = grepl("^8800", ops_ko),
#>   ops_permanent_dialysis = grepl("^53995|^55492|^5392", ops_ko)
#>
#> )
#> }
#>
#> # Aufbereitung ----
#>
#> da |> auswahl() |>
#>   aufbereitung() |>
#>   anreicherung() |>
#>   filter(ops_immuneadsorption | ops_plasmasep) |>
#>   write_parquet("cache/auswertungsdaten.parquet")
#>
#> # Auswertung ----
#>
#> da_out <- read_parquet("Output/auswertungsdaten.parquet")
#>
#> # Klassifikation der Einrichtung
#> zentrum <- da_out |> group_by(berichtsjaehr, ik) |>
#>   summarize(
#>     n_zentrum = n(),
#>     n_iad = sum(ops_immuneadsorption),
#>     n_pe = sum(ops_plasmasep),
#>     .groups = "drop"
#>   ) |> mutate(erfahrung = ntile(n_zentrum, 4))
#>
#> da_out <- da_out |>
#>   left_join(zentrum, by = c("ik", "berichtsjaehr")) %>%
#>   mutate(death = (entl_grd == 7) #| permanent_dialyse
#>   )
#>
#> da_out |> write_parquet("Output/auswertungsdaten.parquet")
#>
#>
#> ## Tabellenausgabe ----
#>
#> da_out |> group_by(ops_plasmasep, ops_immuneadsorption) |>
#>   summarize(n = n(), endpt = sum(death), .groups = "drop"
#>   ) |> fwrite("Output/auswertung1.csv", sep = ";")
#>
#>
#> ## Modell ----
#>
#> ### Vorbereitung der Daten
#>
#> da_mod <- da_out |> mutate(
#>   pe_vs_iad = ifelse(ops_plasmasep & !ops_immuneadsorption, 1, 0),
#>   alter_10 = alter / 10,
#>   big_center = ifelse(erfahrung > 3, 1, 0),
#>   icd_ANCA_overlap = ifelse(icd_ANCA_overlap, 1, 0)

```

```

#> )
#>
#> ### Regression festlegen und kalkulieren
#> regression <- function(outcome){
#>   glm(formula(
#>     paste0(outcome, "~ pe_vs_iad+alter_10+big_center+icd_ANCA_overlap")),
#>     data = da_mod, family = "binomial")
#> }
#>
#> ### Ausgabe
#> model <- regression("death")
#> jtools::summ(model, confint = TRUE) |>
#>   capture.output(file = "Output/model_output.txt")
#>
#>
#> # Succeeded in ~ 9:48 min
#> # Succeeded in ~10:18 min

rm(list = ls()); gc()

```

```

#>          used (Mb) gc trigger (Mb)    max used (Mb)
#> Ncells 3230772 172.6  11843044 632.5  18504754 988.3
#> Vcells 14037980 107.2   628171416 4792.6 6593431809 50303.9

```

### 5.3.5 Auswertung

```

tic()
DF <- read_parquet("cache/auswertungsdaten.parquet")
toc()
obj_size(DF)

str(DF)

```

```

#> 0.021 sec elapsed
#> 416.81 kB
#> tibble [17,109 x 21] (S3: tbl_df/tbl/data.frame)
#>  $ berichtsjahr      : int  [1:17109] 2011 2011 2011 2011 2011 2011 2011 2011 2011 2011 2011 2011 ...
#>  $ kh_land           : int  [1:17109] 4 9 3 3 3 3 4 4 12 8 8 ...
#>  $ kh_typ_gem3       : int  [1:17109] 2 3 2 2 2 2 2 2 3 1 1 ...
#>  $ sex               : chr   [1:17109] "w" "w" "w" "m" ...
#>  $ alter             : int  [1:17109] 30 58 21 71 25 50 45 30 2 67 ...
#>  $ aufn_anl         : chr   [1:17109] "N" "E" "N" "N" ...
#>  $ cm_vol            : num  [1:17109] 2365 7498 2902 7570 2257 ...
#>  $ icd_hd3           : chr   [1:17109] "A04" "K56" "G35" "C90" ...
#>  $ icd_hd4           : chr   [1:17109] "A043" "K565" "G350" "C900" ...
#>  $ icd_hd            : chr   [1:17109] "A043" "K565" "G350" "C9000" ...
#>  $ tage              : int  [1:17109] 13 15 18 5 10 10 19 4 10 14 ...
#>  $ entl_grd          : int  [1:17109] 1 1 1 1 1 1 1 1 1 1 ...
#>  $ ik                : int  [1:17109] 260400184 260912149 260310209 260310209 260310209 260400184 ...
#>  $ icd_nd            : chr   [1:17109] "B962; 0988; 0092; 0990; D593; Z290;" "K564; K760; I1000; I ...
#>  $ ops_ko            : chr   [1:17109] "882004; 8800c1; 8930;" "882000; 546920; 546900; 545420; 32 ...
#>  $ icd_ANCA_overlap  : logi   [1:17109] FALSE FALSE FALSE FALSE FALSE FALSE ...
#>  $ icd_bleeding      : logi   [1:17109] FALSE FALSE FALSE FALSE FALSE FALSE ...
#>  $ ops_immuneadsorption : logi   [1:17109] FALSE FALSE FALSE FALSE FALSE TRUE ...
#>  $ ops_plasmasep     : logi   [1:17109] TRUE TRUE TRUE TRUE TRUE FALSE ...

```



```
#> $ ops_transfusion      : logi [1:17109] FALSE FALSE FALSE FALSE FALSE FALSE ...
#> $ ops_permanent_dialysis: logi [1:17109] FALSE FALSE FALSE FALSE FALSE FALSE ...
```

```
tic()
zentrum <- DF |> group_by(berichtsjaehr, ik) |>
  summarize(
    n_zentrum = n(),
    n_iad = sum(ops_immuneadsorption),
    n_pe = sum(ops_plasmasep),
    .groups = "drop"
  ) |> mutate(erfahrung = ntile(n_zentrum, 4))

DF <- DF |>
  left_join(zentrum, by = c("ik", "berichtsjaehr")) |>
  mutate(death = (entl_grd == 7))

toc()
zentrum |> head() |> print_table()
```

**5.3.5.1 Weitere Anreicherung** #> Error in print\_table(head(zentrum)): could not find function "print\_table"

#> 0.045 sec elapsed

#### 5.3.5.2 Tabellen Outcome vs ANCA Overlap

```
tic()
DF |> group_by(icd_ANCA_overlap) |>
  summarize(
    n = n(),
    death = sum(death)
  ) |> mutate(freq = round(death / n, 4)) |>
  print_table()
```

#> Error in print\_table(mutate(summarize(group\_by(DF, icd\_ANCA\_overlap), : could not find function "print\_table"

```
toc()
```

#> 0.002 sec elapsed

#### IAD vs PSE

```
tic()
DF |> group_by(ops_plasmasep, ops_immuneadsorption) |>
  summarize(
    n = n(),
    death = sum(death)
  ) |> mutate(freq = round(death / n, 4)) |>
  print_table()
```

#> Error in print\_table(mutate(summarize(group\_by(DF, ops\_plasmasep, ops\_immuneadsorption), : could not find function "print\_table"

```
toc()
```

#> 0.001 sec elapsed

Outcome abhängig davon ob IAD im Zentrum behandelt wird

```
tic()
DF |>
  mutate(iad_centre = (n_iad > 0)) |>
  group_by(iad_centre) |>
  summarize(
    n = n(),
    endpt = sum(death)
  ) |> mutate(freq = round(endpt / n, 4)) |>
  print_table()
```

#> Error in print\_table(mutate(summarize(group\_by(mutate(DF, iad\_centre = (n\_iad > : could not find function "print\_table"

```
toc()
```

#> 0.002 sec elapsed

PE vs IAD in IAD centres

```
tic()
DF %>%
  mutate(iad_centre = (n_iad > 0)) %>%
  filter(iad_centre) %>%
  group_by(ops_plasmasep, ops_immuneadsorption) %>%
  summarize(
    n = n(),
    death = sum(death)
  ) %>% mutate(freq = round(death / n, 4))
toc()
```

```
#> # A tibble: 2 x 5
#> # Groups:   ops_plasmasep [2]
#>   ops_plasmasep ops_immuneadsorption      n death   freq
#>   <lgl>         <lgl>                <int> <int> <dbl>
#> 1 FALSE       TRUE                 8309   131 0.0158
#> 2 TRUE        FALSE                 6438   101 0.0157
#> 0.033 sec elapsed
```

Struktur in IAD-IKs

```
DF %>%
  mutate(
    alter = paste0("Ü", (round(alter / 30)) * 30),
    iad_centre = (n_iad > 0)
  ) %>%
  group_by(iad_centre, ops_plasmasep, ops_immuneadsorption) %>%
  summarize(
    n = n(),
    Ü0 = sum(alter == "Ü0"),
    Ü30 = sum(alter == "Ü30"),
    Ü60 = sum(alter == "Ü60"),
    Ü90 = sum(alter == "Ü90"),
    W = sum(sex == "w"),
    large = sum(erfahrung > 3),
    anca = sum(icd_ANCA_overlap),
```

```
#ct, boncho, kidney_biopsy, komplikationen ...
```

```
tage_median = median(tage),
tage_mean = mean(tage),
tage_var = var(tage),
tage_sd = sd(tage),
tage_iqr = IQR(tage, na.rm = TRUE, type = 7),
ops_transfusion = sum(ops_transfusion),
verstorben = sum(entl_grd == 7),
.groups = "drop"
)
```

```
#> # A tibble: 3 x 18
#>   iad_centre ops_plasmasep ops_immuneadsorption     n    Ü0    Ü30    Ü60    Ü90
#>   <lgl>      <lgl>        <lgl>              <int> <int> <int> <int> <int>
#> 1 FALSE    TRUE        FALSE              2362   16   768  1377  201
#> 2 TRUE     FALSE       TRUE               8309  357  3104  4412  436
#> 3 TRUE     TRUE        FALSE              6438  126  2220  3713  379
#> # i 10 more variables: W <int>, large <int>, anca <int>, tage_median <dbl>,
#> #   tage_mean <dbl>, tage_var <dbl>, tage_sd <dbl>, tage_iqr <dbl>,
#> #   ops_transfusion <int>, verstorben <int>
```

### 5.3.5.3 Modelle Vorbereitung auf Regression:

```
tic()
DF_mod <- DF %>% mutate(
  pe_vs_iad = ifelse(ops_plasmasep & !ops_immuneadsorption, 1, 0),
  alter_10 = alter / 10,
  big_center = ifelse(erfahrung > 3, 1, 0),
  icd_ANCA_overlap = ifelse(icd_ANCA_overlap, 1, 0)
)
toc()
```

```
#> 0.005 sec elapsed
```

#### Ausführung

GLM, um herauszufinden, wie der Tod des Patienten von der Diagnose (PSE vs IAD), begleitenden Symptomen (ANCA), Fokus der Klinik und Alter abhängt.

```
tic()
regression <- function(outcome){
  glm(formula(
    paste0(outcome, "~ pe_vs_iad+alter_10+big_center+icd_ANCA_overlap")),
    data = DF_mod, family = "binomial")
}
```

```
#regression("ops_transfusion") %>% summary()
#regression("ops_permanent_dialysis") %>% summary()
#regression("icd_bleeding") %>% summary()
```

```
# outcome dauerhaft Dialyse oder tod
regression("death") %>% summary()
toc()
```

```
#>
```

```

#> Call:
#> glm(formula = formula(paste0(outcome, "~ pe_vs_iad+alter_10+big_center+icd_ANCA_overlap")),
#>       family = "binomial", data = DF_mod)
#>
#> Deviance Residuals:
#>      Min       1Q   Median       3Q      Max
#> -0.7764  -0.1964  -0.1382  -0.0996   3.7645
#>
#> Coefficients:
#>              Estimate Std. Error z value Pr(>|z|)
#> (Intercept)    -5.648908   0.260651  -21.672  <2e-16 ***
#> pe_vs_iad       0.002597   0.118155   0.022   0.982
#> alter_10        0.469547   0.038614  12.160  <2e-16 ***
#> big_center      -1.485577   0.117496 -12.644  <2e-16 ***
#> icd_ANCA_overlap -11.091116 227.659435  -0.049   0.961
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> (Dispersion parameter for binomial family taken to be 1)
#>
#>      Null deviance: 3148.8  on 17108  degrees of freedom
#> Residual deviance: 2774.6  on 17104  degrees of freedom
#> AIC: 2784.6
#>
#> Number of Fisher Scoring iterations: 13
#>
#> 0.093 sec elapsed

```

## 6 Abschluss

- Mustersyntax (ggf. für Fortgeschrittene)
- interne Datenoperationen?

Mögliche **weiterführende Themen**

- Version Control mit **git**
- Workflow Tools (bspw. targets)
- Python Alternativen (hier gibt es mehr!)